

# Database After Graduation

By Narges Yarahmadi Feb 2026



## Production SQL &

*"Why your beautifully modeled schema is crying at 3 AM"*

“**"Have you ever written a query that worked perfectly locally, then brought down staging?"**

### ⚡ REALITY CHECK

Most database problems aren't syntax errors—they're performance errors. You don't know your query until you know its execution plan.

### SESSION GOAL

By the end of tonight, you'll stop thinking in ~~SQL-syntax~~ and start thinking in:



Page Fetches



Index Traversals



Join Algorithms

# THE MENTAL MODEL SHIFT



## BEFORE GRADUATION

*"Does my query return the right data?"*

*"Is my syntax correct?"*

*"Did I normalize to 3NF?"*

---

FOCUS: CORRECTNESS & STRUCTURE



## AFTER GRADUATION

- ✓ "How many rows did this query actually read vs return?"
- ✓ "Is this using an index or scanning a table?"
- ✓ "What's the latency impact at peak traffic?"
- ✓ "Can this query take a lock that blocks writes?"

---

FOCUS: COST & CONCURRENCY



### KEY INSIGHT

In production, you don't write queries.  
You design access paths.



### ANALOGY

Writing SQL without understanding execution plans is like giving driving directions without knowing which roads are under construction, one-way, or have tolls.

# PRODUCTION SQL $\neq$ TEXTBOOK SQL

## 📖 WHAT TEXTBOOKS TEACH

```
SELECT * FROM orders WHERE customer_id = 123;  
-- Clean. Simple. Works.
```

## 🏢 WHAT PRODUCTION TEACHES

```
SELECT  
  o.id,  
  o.total_amount,  
  c.email,  
  oi.product_name  
FROM orders o  
INNER JOIN customers c ON o.customer_id = c.id  
INNER JOIN order_items oi ON o.id = oi.order_id  
WHERE o.created_at >= '2024-01-01'  
      AND c.tier = 'premium'  
ORDER BY o.created_at DESC  
LIMIT 50;
```

### 🔍 The Hidden Questions

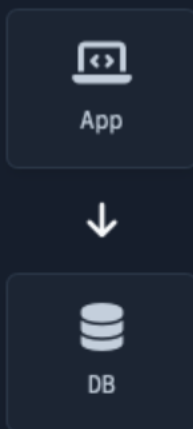
- ❓ Does `customer_id` have an index? What kind?
- ❓ Is `created_at` indexed? Is it selective enough?
- ❓ How does the join order get determined?
- ❓ Is `LIMIT 50` applied before or after sorting?
- ❓ What happens when premium customers are 80% of your user base?

### PRODUCTION MINDSET

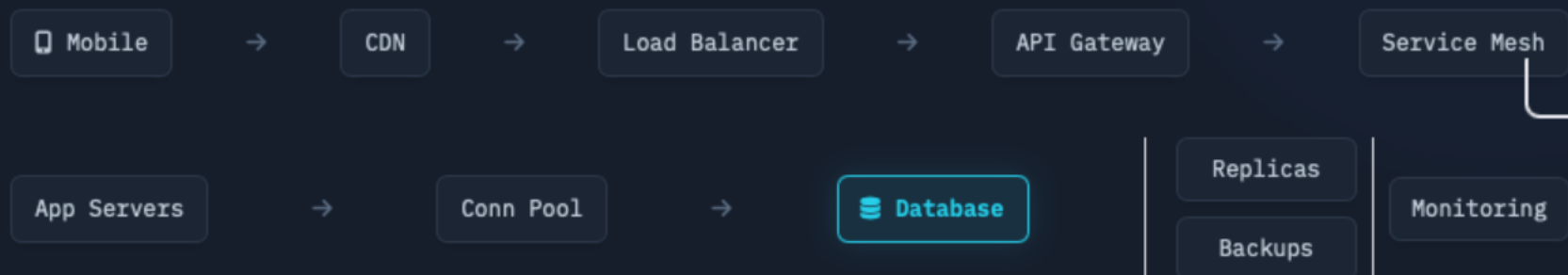
Every query is a trade-off. You're not just retrieving data—you're consuming resources that other queries need.

# THE ARCHITECTURE REALITY CHECK

## 🎓 CLASS PROJECTS



## 🔧 REAL ARCHITECTURE



## 🔥 WHERE QUERIES ACTUALLY BREAK

- ❌ N+1 in ORMs  
100 queries instead of 1
- ❌ Pool Exhaustion  
Slow queries hold conns
- ❌ Replica Lag  
Read/write inconsistency
- ❌ Lock Contention  
Writes blocked by reads

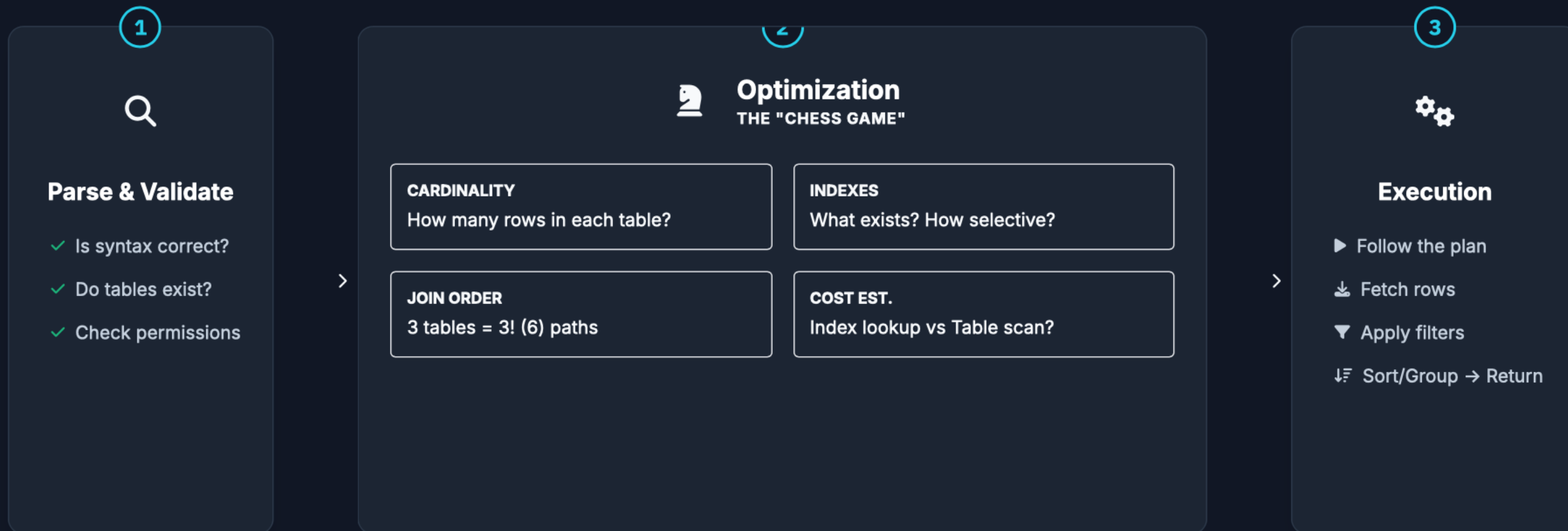
## INDUSTRY EXAMPLE: SHOPIFY

At Shopify, a single unindexed query during Black Friday doesn't just fail—it **cascades**.

Pool fills → Health checks fail → Load balancer kills instance → Rollback

*Your query is never "just a query."*

# THE DATABASE'S DECISION TREE



## CRITICAL FACT



The optimizer is **guessing**. It uses statistics that might be stale. It doesn't know that your status column is 99.9% 'active' —it just sees two distinct values.



# EXPLAIN | YOUR PRODUCTION SUPERPOWER

*"EXPLAIN is not a debugging tool. It's a design tool."*

## ✖ THE BEFORE STATE

Write Query → Deploy → **Pager Duty 2 AM**

## ✔ THE AFTER STATE

EXPLAIN → Add Index → **Sleep**

## 🗉 WHAT EXPLAIN ACTUALLY TELLS YOU

| COLUMN | WHAT IT REALLY MEANS                                    |
|--------|---------------------------------------------------------|
| type   | Is this efficient or are we suffering?                  |
| key    | Which index (if any) is actually used                   |
| rows   | MySQL's estimated rows to examine                       |
| Extra  | "Using where", "Using filesort" (The performance story) |

## ☰ The Three Scans

**const** **Perfect**  
Primary key lookup. 1 row.

**ref** **Good**  
Non-unique index lookup. Small set.

**ALL** **Bad**  
Full table scan. Get coffee.

## INDUSTRY RULE

If you see **ALL** on a table > 10,000 rows in a hot path, you have a production incident waiting to happen.

# LIVE EXPLAIN WALKTHROUGH

Scenario: E-commerce order search

## ❌ QUERY 1: THE "WORKS ON MY MACHINE"

```
SELECT * FROM orders
WHERE customer_email LIKE '%gmail.com';
```

### EXPLAIN Result

type: ALL

rows: 5,000,000

**Why:** Leading wildcard % prevents B-Tree traversal. Full table scan inevitable.

## ✅ QUERY 2: THE "FIXED" VERSION

```
SELECT * FROM orders
WHERE customer_email = 'user@gmail.com';
```

### EXPLAIN Result

type: ref

key: idx\_email | rows: 1

**Result:** Direct index lookup. Fast and scalable.

## ⚠️ QUERY 3: THE HIDDEN TRAP & FIX

```
// The Trap
SELECT * FROM orders
WHERE DATE(created_at) = '2024-02-11';
```

```
// The Fix
SELECT * FROM orders
WHERE created_at >= '2024-02-11'
AND created_at < '2024-02-12';
```

### BEFORE

type: ALL 🚩

### AFTER

type: range

**Why:** Wrapping a column in a function (DATE()) hides the value from the index.

### KEY LESSON

MySQL isn't intelligent. It follows rules. Learn the rules.



# INDEXING STRATEGIES THAT SCALE

## RULE 2 THE COMPOSITE INDEX RULE

```
SELECT * FROM orders
WHERE status = 'active' AND created_at > '2024-01-01';

-- ❌ Bad Index: (status)
-- Why: Filters to 'active', then scans ALL active rows to check date

-- ✅ Good Index: (status, created_at)
-- Why: Jumps to 'active' section, then range scan on created_at
```

## RULE 3 THE LEFTMOST PREFIX TRAP

Index Definition: A B C

WHERE A = ?



WHERE B = ?



WHERE A=? AND B=?



WHERE C = ?



## RULE 4 COVERING INDEXES (THE HACK)

```
SELECT id, email FROM users WHERE status = 'active';

-- Index: (status, email, id)
-- EXPLAIN Extra: "Using index"
-- Result: Database NEVER touches the table. Reads only from index tree.
```

### THE MISCONCEPTION

"Indexes make queries faster."

### THE TRUTH

Some indexes make *some* queries faster.

Every index makes **writes slower** and consumes disk space.

### 1 Query-First Design

#### ● DON'T ASK

"What indexes should I add?"

#### DO ASK

"What queries will run?"

#### ● THEN ASK

"What index makes THAT query fast?"

### 🏢 INDUSTRY PRACTICE

At companies like **Uber/Lyft**, critical queries are often index-only scans. The database never touches the actual table rows to serve the request.

# READ-HEAVY VS WRITE-HEAVY | THE TRADEOFF MATRIX

| FEATURE    | 👁 READ-HEAVY                | ✍ WRITE-HEAVY                   |
|------------|-----------------------------|---------------------------------|
| INDEXES    | Many, composite, covering   | Minimum required, narrow        |
| SCHEMA     | Denormalized, pre-joined    | Normalized, minimal constraints |
| PATTERN    | Complex SELECTs             | Simple INSERTs / UPDATEs        |
| BOTTLENECK | I/O, join costs             | Lock contention, disk sync      |
| EXAMPLE    | Analytics dashboard, Search | IoT sensor data, Logs           |

## 🔍 READ-HEAVY STRATEGY

- ✓ Add indexes aggressively
- ✓ Consider summary tables
- ✓ Cache aggressively (Redis)
- ✓ Use read replicas

## 📄 WRITE-HEAVY STRATEGY

- ✓ Minimal indexes (~2x cost/idx)
- ✓ Batch writes
- ✓ Avoid FK constraints (app integrity)
- ✓ Partition by time

## REAL WORLD: UBER



**Trip History:** A schizophrenic table.

- Write-heavy for active GPS
- Read-heavy for rider history

*Solution: Different indexes/tables based on state.*

# THE READ PATH | WHAT HAPPENS WHEN YOU SELECT

01

## </> Parser

"What query is this?"

02

## 📋 Preprocessor

"Do tables/columns exist?"

03

## 👤 Optimizer

"How should I get this?"

- Check statistics
- Consider indexes
- Calculate costs
- Choose plan

## ⚠️ THE HIDDEN COST

Each step adds latency. A query that returns **10 rows** might read:

10,000 index entries  
+ 100 data pages

06

## 🚀 Server

Format result → Send to client

05

## 💾 Storage Engine

- Navigate B-tree indexes
- Locate page on disk
- Load into buffer pool
- Return row

04

## ☰ Executor

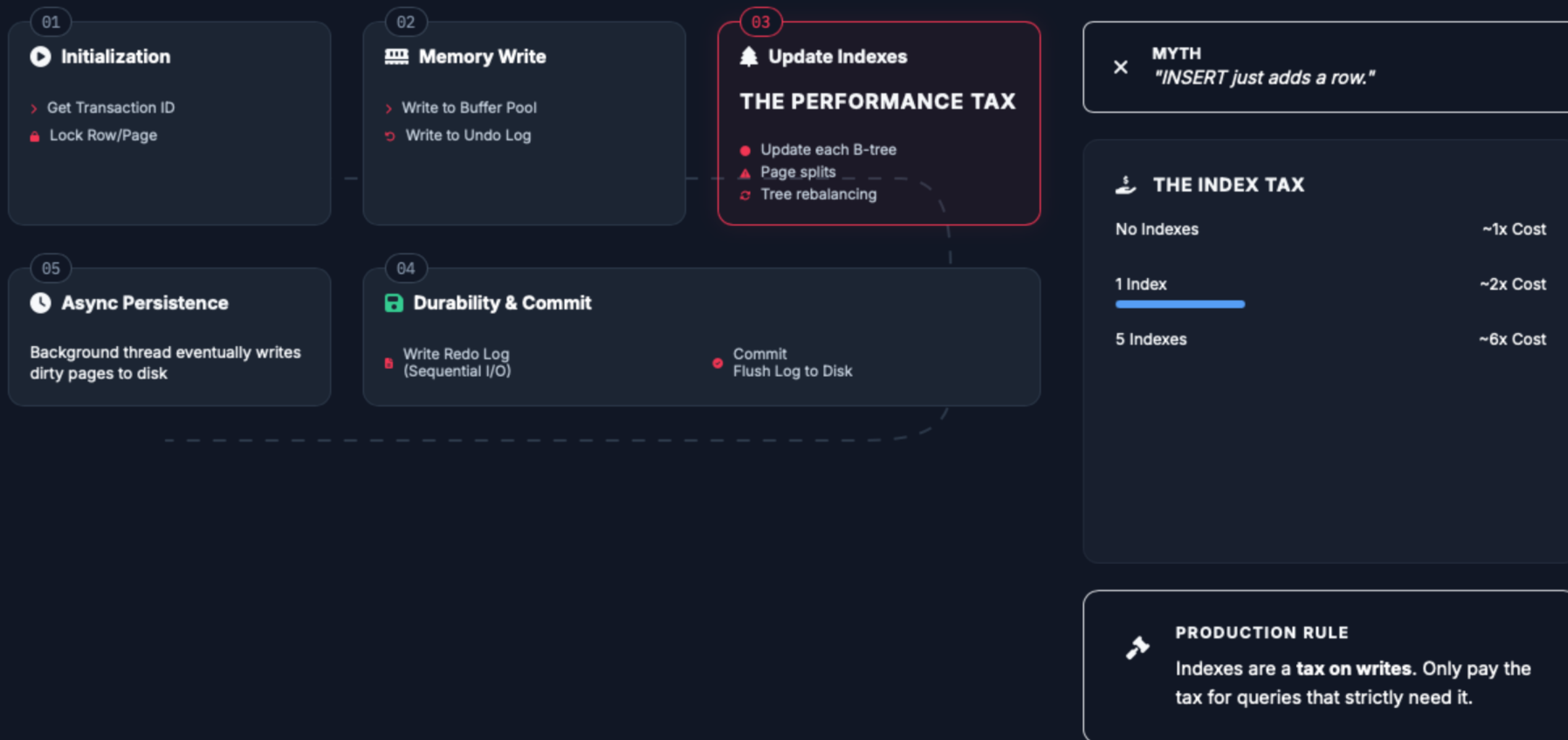
"Follow the plan"



## KEY INSIGHT

Most of the cost isn't in the network—it's in traversing data structures and waiting for disk I/O.

# THE WRITE PATH | WHAT HAPPENS WHEN YOU INSERT



# LOCKING | THE SILENT PRODUCTION KILLER

## ☰ LOCK TYPES YOU'LL ACTUALLY ENCOUNTER



### Row Locks

- > `SELECT ... FOR UPDATE` "I'm changing these, back off."
- > `UPDATE` Automatically locks modified rows.



### Gap Locks

- > "Don't insert into this range while I'm reading."
- > Prevents phantom reads, but blocks inserts.



### Table Locks

- > Triggered by DDL (`ALTER TABLE`) or backups.
- > Usually a sign of trouble in OLTP systems.

NEW GRADS

Atomicity

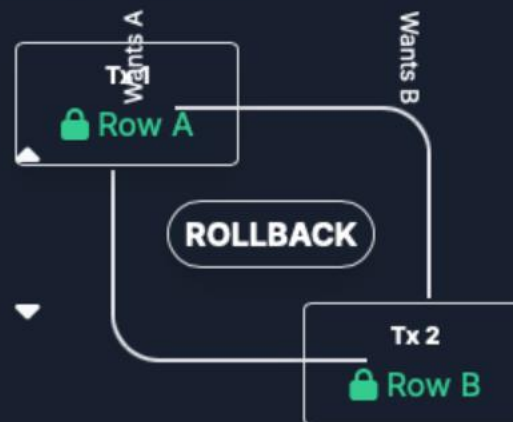


PRODUCTION

Locks



## THE DEADLOCK TRIANGLE



### INDUSTRY WISDOM

"Deadlocks aren't bugs; they're expected behavior. Handle them. Retry."

# CASE STUDY | THE "SIMPLE" DASHBOARD

## THE SCENARIO

Startup grows from **100** to **100,000** customers.

### REQUIREMENT

"Simple admin dashboard: Show recent orders with customer details."

## POST-MORTEM FINDINGS

🔍 Scanning entire orders table

⬇️ Sorting **5M rows** to return 100

🚫 Blocking inserts during sort (filesort)

"Queries don't break when you write them. They break when your data grows."



### Week 1: The Honeymoon

LATENCY: 5MS

```
SELECT * FROM orders ORDER BY created_at DESC LIMIT 100;
```

😊 100 rows returned. Instant. Everyone's happy.



### Week 20: Feature Creep

LATENCY: 450MS

```
SELECT o.*, c.name, c.email FROM orders o JOIN customers c ON o.customer_id = c.id ORDER BY o.created_at DESC LIMIT 100;
```

Still 100 rows returned. But now reading millions. No index on created\_at = filesort (temp table).



### Week 40: The 2 AM Outage

TIMEOUT

📶 Connection pool fills

📶 API endpoints degrade

#### THE FIX:

```
-- Index on (created_at) covering customer_id -- Deferred Join (filter IDs first, then join)
```



# CASE STUDY | EVENT TICKETING (WRITE HELL)

## THE SCENARIO

Popular concert tickets go on sale. High concurrency, limited inventory.

### TRAFFIC

100k Users  
Simultaneous

### INVENTORY

10k Tickets  
Strict Limit

## THE REALITY CHECK

Race Condition:

👥 100 users check availability at once

👁️ All see 1 ticket left

🛒 All insert orders & decrement

💣 **Result: -99 Tickets** 🤯

KEY PRINCIPLE

"Sometimes the database primitive is the solution, not application logic."



## The Naive Approach (Application Logic)

BUGGY

```
-- 1. Check availability (Read) SELECT available FROM inventory WHERE event_id = 1; -- 2. If > 0, Insert & Update (Write) INSERT INTO orders (user_id, event_id) VALUES (123, 1); UPDATE inventory SET available = available - 1...
```

- ✖ Not atomic. Time gap between Read and Write allows overselling.



## The Production Solution (Atomic Update)

SAFE

```
-- Atomic decrement with condition in ONE statement UPDATE inventory SET available = available - 1 WHERE event_id = 1 AND available > 0; -- Check affected rows: 1 = Success, 0 = Sold Out
```

Database handles locking internally. No gap between check and update.



## Index Strategy

✓ Primary Key on event\_id

Essential for row lock location.

✗ Index on available

Low selectivity, high write overhead. Avoid.



# QUERY OPTIMIZATION HEURISTICS

## ▼ STRATEGY: PUSH DOWN FILTERS

-- ❌ BAD: Join massive tables, then filter result

```
SELECT * FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE c.tier = 'premium' AND o.amount > 1000;
```

-- ✅ GOOD: Filter first to reduce join set size

```
SELECT * FROM
  (SELECT * FROM customers WHERE tier = 'premium') c
JOIN
  (SELECT * FROM orders WHERE amount > 1000) o
ON o.customer_id = c.id;
```

## 🕒 STRATEGY: AVOID FUNCTIONS ON COLUMNS

-- ❌ BAD: Disables Index Usage (Full Table Scan)

```
WHERE YEAR(created_at) = 2024
```

-- ✅ GOOD: Enables Range Scan on Index

```
WHERE created_at >= '2024-01-01'
AND created_at < '2025-01-01'
```

## 📊 The 80/20 Rule of Performance

### 1. Reduce Rows Examined

If you return 10 rows, don't examine 10,000. Each ALL scan is a potential incident.

### 2. Push Down Filters

Filter rows as early as possible in the execution plan before expensive joins.

### 3. Avoid Functions on Columns

Wrapping a column in a function hides the data distribution from the optimizer.

### 4. Limit Early

Don't join 1M rows then take the top 10. Subquery the limit, then join.

### 5. Know Your Cardinalities

Index on gender (2 values) is useless. Index on email (unique) is highly selective.

# THE CONNECTION POOL | WHERE QUERIES WAIT TO DIE

## ARCHITECTURE REALITY



## THE POOL AS CONSTRAINT

### POOL SIZE

10 - 30

Connections per app instance

### CONCURRENCY RULE

1 Connection = 1 Query at a time

### THE EQUATION

Slow Query =  
WHY INDEXES ARE NON-NEGOTIABLE

⚠ Occupied Connection =  
Pool Exhaustion

A missing index doesn't just slow down one query. It creates system-wide instability.

## THE CASCADE OF FAILURE



### Latency Spike

Query takes 5 seconds instead of 50ms



### Resource Starvation

Connection held for 5s. All connections become busy.



### Queue Pile-up

New requests queue up. Health check probes time out.



### System Failure

Load balancer marks instance unhealthy. Traffic shifts. Cascade repeats.

# MONITORING | WHAT TEAMS ACTUALLY WATCH

>\_ The Dashboard Nobody Builds But Everyone Needs



## Query Performance

Execution efficiency

- **P99 latency** by query signature
- Rows examined vs rows returned Ratio
- Full table scans / minute
- Temporary table creations (on disk)



## Index Usage

Schema health

- **Unused indexes** (write tax, no read benefit)
- Redundant indexes (`idx_a`, `idx_a_b`)
- Index cardinality vs actual selectivity

`idx_users_email`

USAGE: 0



## Lock Monitoring

Concurrency issues

- **Lock wait timeouts** (Application errors)
- Deadlock frequency
- Long-running transactions (>1s)

Contention Level



## Resource

System health

- Buffer pool hit ratio **>99%**
- Disk I/O utilization (IOPS)
- Connection usage / Max connections

CPU

RAM

DISK



### INDUSTRY PRACTICE

At scale, you don't optimize individual queries—you optimize **query families** and monitor regressions.

# PRODUCTION CHECKLIST | BEFORE YOU DEPLOY

## 🔒 THE "AM I GOING TO GET PAGED?" CHECKLIST

### </> Query Level

- ☒ Did I run EXPLAIN?
- ☒ Is type better than ALL on large tables?
- ☒ Am I using functions on indexed columns? (e.g. YEAR(date))
- ☒ Is my ORDER BY covered by an index?
- ☒ Am I joining on columns with compatible types?

### 📦 Index Level

- ☒ Does each index serve a specific query pattern?
- ☒ Is the **most selective** column first in composite indexes?
- ☒ Are there redundant indexes I can remove? (idx\_a\_b covers idx\_a)
- ☒ Have I considered **covering indexes** for hot queries?

### ⚖️ System Level

- ☒ What's the impact during peak writes?
- ☒ Can this query cause lock contention?
- ☒ Will this work on replicas with lag?
- ☒ **What's the rollback plan?**

## 👥 CULTURE SHIFT

### OLD WAY



*"It works on my machine."*

🚫 Not a deployment approval

### NEW WAY



"EXPLAIN shows index-only scan with const type."

✅ Ready for Production

*"Reliability is not an accident. It's a checklist."*

# TAKE-HOME | MENTAL MODELS

## ☰ CORE DATABASE REALITY



### 1. The Query as Resource Consumer

Every query consumes CPU, memory, I/O, and locks. You are **sharing these resources** with every other query.



### 2. The Index as Access Path

Indexes aren't just "performance" features—they are data structures (B-Trees) that provide **alternative ways** to find rows.



### 3. The Execution Plan as Contract

Your query doesn't do what you wrote. It does what the **optimizer decided** based on statistics. Always verify it.

## ☰ OPERATIONAL REALITY

### 04 Production ≠ Dev

DEV 1K rows, No load

PROD 10M rows, 500 QPS

*"Fragmented indexes and concurrent queries change everything."*

### 05 THE TRADEOFF MINDSET

Every database decision is a trade-off.

|              |   |                   |
|--------------|---|-------------------|
| Fast Reads?  | → | Slower Writes     |
| Low Latency? | → | Higher Memory     |
| Consistency? | → | Less Availability |



# COURSE WRAP-UP | CLOSING THOUGHTS

## OVERVIEW

## The Real Curriculum



### WEEK 1

How to build databases.



### WEEK 2 (TODAY)

How to not break databases.



### WEEK 3

How to scale databases.



### WEEK 4

How to evolve databases.



## FINAL THOUGHT

Production Wisdom



```
# quote_of_the_day.txt
```

```
"The best query isn't the one  
with perfect syntax—it's the one  
that runs 10,000 times a day and  
nobody notices."
```

```
✓ Deployment successful
```

?

## Questions?

Open floor for discussion



# APPENDIX | QUICK REFERENCE CARDS

## 🔍 EXPLAIN Output Cheat Sheet

Target: type != ALL

| TYPE                      | BADNESS | MEANING                 |
|---------------------------|---------|-------------------------|
| <code>system/const</code> | 😊       | 1 row, primary key      |
| <code>eq_ref</code>       | 😊       | JOIN using PK           |
| <code>ref</code>          | 😐       | Non-unique index lookup |
| <code>range</code>        | 😐       | Index range scan        |
| <code>index</code>        | 😞       | Full index scan         |
| <b>ALL</b>                | 💀       | Full table scan         |

# APPENDIX | QUICK REFERENCE CARDS



## INDEX DESIGN RULES

Best practices for B-Tree optimization



### Target Key Clauses

Index columns used in WHERE, JOIN, and ORDER BY.



### Selectivity First

Place the most selective column (highest cardinality) first in composite indexes.



### Keep It Lean

Limit composite indexes to 3-4 columns max to reduce storage and write overhead.



### The Write Tax

Always consider write frequency (INSERT/UPDATE/DELETE) before adding an index.



## WHEN TO DENORMALIZE

Trading space for speed



### Join Hell

When critical reporting queries require joining **5+ tables**.



### Pre-Aggregation

When sum/count/avg calculations over large datasets are needed instantly.



### Extreme Read Ratio

When Read:Write ratio exceeds **100:1** (e.g., product catalog, user profile).



**Warning:** Denormalization introduces data anomalies. You must handle data consistency in the application layer.